# Graphs II

## Distance, Constraints, and Logical Structure

King's Competitive Programming Club

Workshop 5

# Workshop Overview

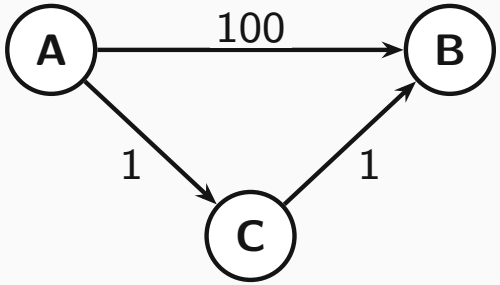


1. Recap: BFS to Weighted Costs
2. Dijkstra's Algorithm
3. 0–1 BFS
4. Path Objectives and Model Choice
5. Conclusion

# Recap: BFS to Weighted Costs



# Recap: BFS Counts Edges, Not Costs




- → Week 4 BFS finds the fewest **edges**.
- → Each edge now has a **weight** (cost), and a path costs the sum of its weights.
- → Here $A \to B$ costs 100, but $A \to C \to B$ costs $1 + 1 = 2$.

We want the minimum total **cost**, not the fewest edges.

# Dijkstra's Algorithm



## Relaxation: Improve a Best-Known Route

- $\rightarrow$ $\mathrm{dist}[v]$ is the cheapest route **found so far** to $v$.
- $\rightarrow$ Start with $\mathrm{dist}[s] = 0$; every other label is $\infty$.
- $\rightarrow$ Relax an edge $u \rightarrow v$ of weight $w$:

$$\mathrm{candidate} = \mathrm{dist}[u] + w.$$

If $\mathrm{candidate} < \mathrm{dist}[v]$, update $\mathrm{dist}[v]$ and push $(\mathrm{dist}[v], v)$. Use a **min-heap**: it removes the smallest distance, not the oldest entry contrary to BFS.

# The Invariant That Makes Dijkstra Safe

**Invariant:** when the current minimum entry for $u$ is popped, $\mathrm{dist}[u]$ is final.

- $\rightarrow$ Any cheaper route to $u$ must leave the popped set through some unpopped vertex $x$.
- $\rightarrow$ With nonnegative weights, its cost up to $x$ is at most its total cost, so $\mathrm{dist}[x] < \mathrm{dist}[u]$.
- $\rightarrow$ Then $x$ would have been popped before $u$: a contradiction.

A negative edge breaks this argument. Do **not** use Dijkstra on negative-weight graphs.

# Dijkstra in Code (C++ and Python)

## C++ (priority_queue)

```
vector<ll> dist(n, INF);
priority_queue<PII, vector<PII>,
               greater<PII>> pq;
dist[src] = 0;
pq.push({0, src});
while (!pq.empty()) {
  auto [d, u] = pq.top();
  pq.pop();
  if (d != dist[u]) continue;
  for (auto [v, w] : adj[u]) {
    if (d + w < dist[v]) {
      dist[v] = d + w;
      pq.push({dist[v], v});
    }
  }
}
```

## Python (heapq)

```
def dijkstra(adj, src):
  dist = [float("inf")] * len(adj)
  dist[src] = 0
  heap = [(0, src)]
  while heap:
    d, u = heappop(heap)
    if d != dist[u]:
      continue
    for v, w in adj[u]:
      nd = d + w
      if nd < dist[v]:
        dist[v] = nd
        heappush(heap, (nd, v))
  return dist
```

$\mathrm{adj}[u]$ stores $(v, w)$ pairs. In C++, `ll` is `long long` and `PII` is `pair<ll,int>`.

## Stale Entries and the Stopping Rule

- → Heap entries are **snapshots** $(\text{distance}, \text{vertex})$. Improving a label pushes a new snapshot; the old one stays.
- → A snapshot with $d \neq \text{dist}[u]$ is **stale**: skip it without scanning neighbours.
- → **One target?** Stop when the target is *popped*, after the stale check.
- → **All distances?** Run until the heap is empty.
- → **Complexity?** $O(V + E \log E)$ time and $O(V + E)$ space.

Do not stop on first discovery or mark a vertex final on insertion.

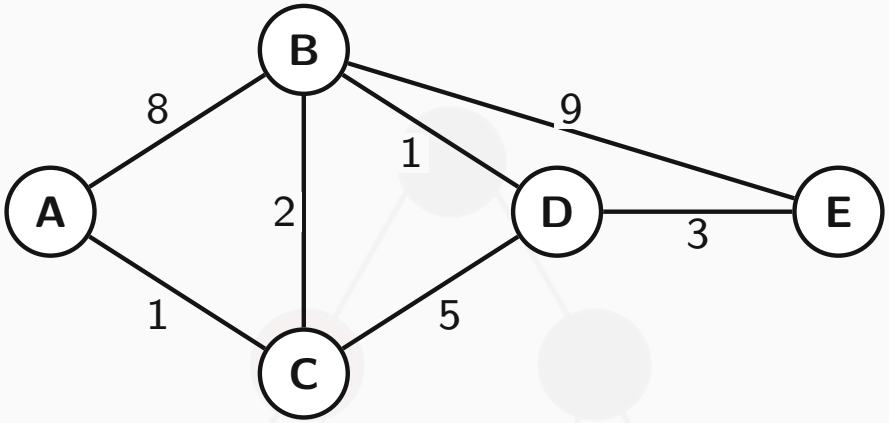
# Live 1 / Problem 1: GFG Dijkstra

**GFG: Dijkstra Algorithm** supplies an **undirected edge list**.

```
class Solution:
    def dijkstra(self, V, edges, src):
        adj = [[] for _ in range(V)]
        for u, v, w in edges:
            adj[u].append((v, w))
            adj[v].append((u, w))
        return dijkstra(adj, src)
```

Reuse the helper from the code slide. The historical title says *adjacency matrix*; use the current signature `V, edges, src`.

# Live 1: a Graph We Can Trace

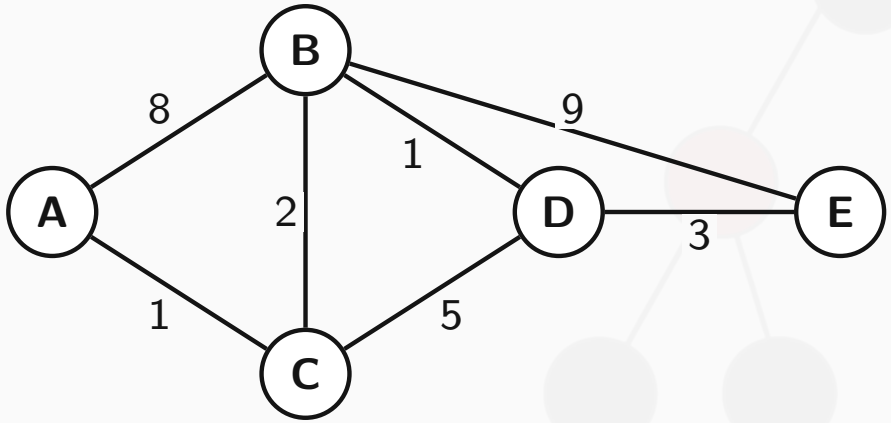



**Undirected.** Source A, target E. Labels A–E map to 0–4.

$\text{edges} = [(0, 1, 8), (0, 2, 1), (1, 2, 2), (1, 3, 1), (2, 3, 5), (1, 4, 9), (3, 4, 3)]$

Predict the order of **valid pops**. An edge weight is not the distance of its endpoint.

# Live 1: Step-by-Step Walkthrough



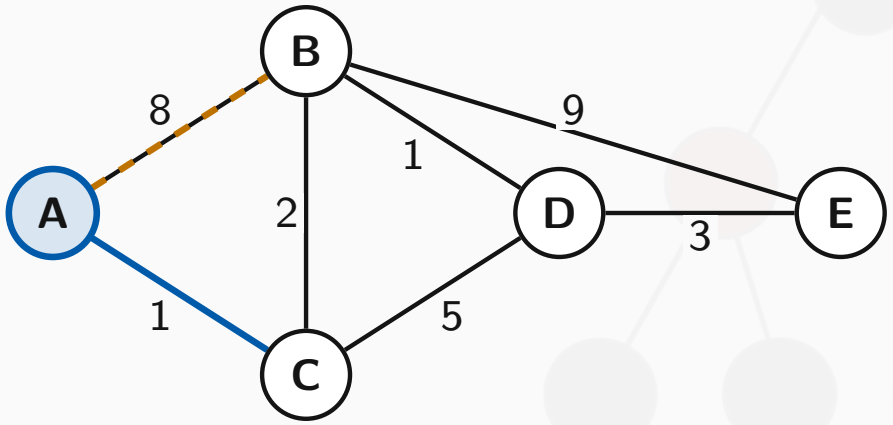

**Initialise**

Push $(0, \mathrm{A})$.

**Heap after this step:**

$[(0, \mathrm{A})]$

# Live 1: Step-by-Step Walkthrough

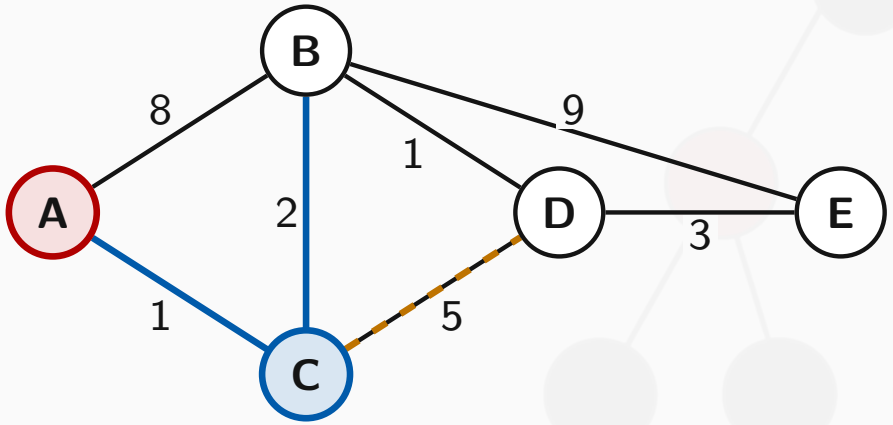



**Pop** $(0, \mathrm{A})$
Improve B to 8 and C to 1.

**Heap after this step:**

$[(1, \mathrm{C}), (8, \mathrm{B})]$

# Live 1: Step-by-Step Walkthrough

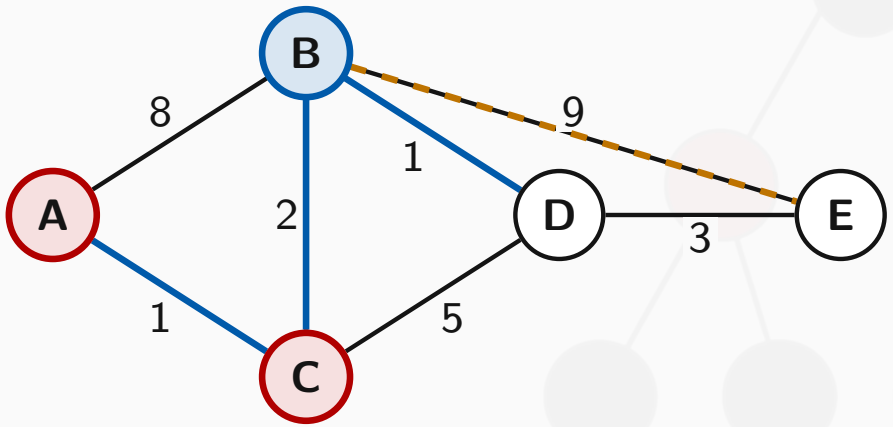



**Pop** $(1, \mathrm{C})$
Improve B to 3 via C, and D to 6.

**Heap after this step:**

$[(3, \mathrm{B}), (6, \mathrm{D}), (8, \mathrm{B})]$

# Live 1: Step-by-Step Walkthrough

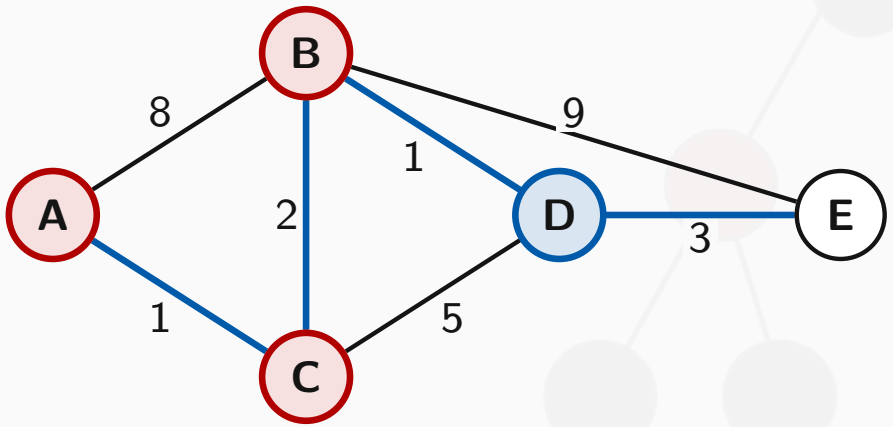



**Pop** $(3, \mathrm{B})$
Improve D to 4 and E to 12.

**Heap after this step:**

$[(4, \mathrm{D}), (6, \mathrm{D}), (8, \mathrm{B}), (12, \mathrm{E})]$

# Live 1: Step-by-Step Walkthrough

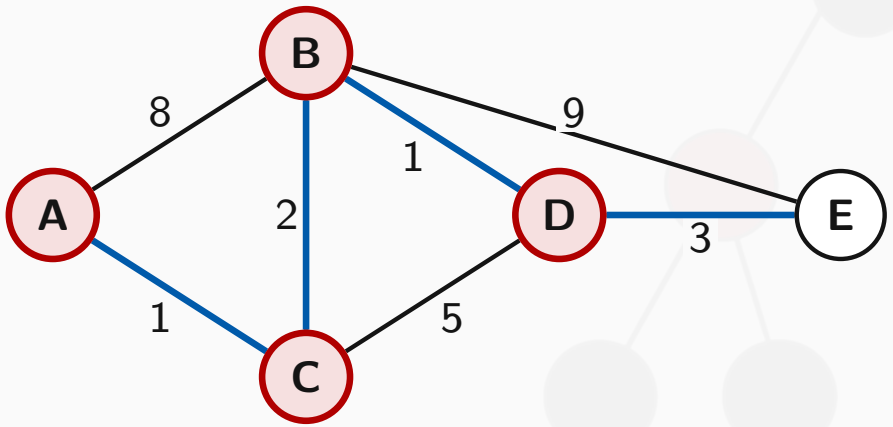



**Pop** $(4, \mathrm{D})$
Improve E to 7 via D.

**Heap after this step:**

$[(6, \mathrm{D}), (7, \mathrm{E}), (8, \mathrm{B}), (12, \mathrm{E})]$

# Live 1: Step-by-Step Walkthrough

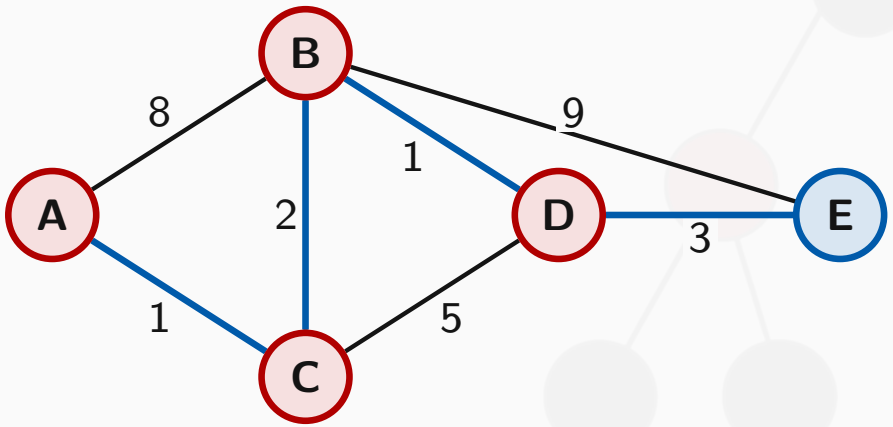



**Pop** $(6, \mathrm{D})$
Stale: $6 \neq 4$. Skip.

**Heap after this step:**

$[(7, \mathrm{E}), (8, \mathrm{B}), (12, \mathrm{E})]$

# Live 1: Step-by-Step Walkthrough

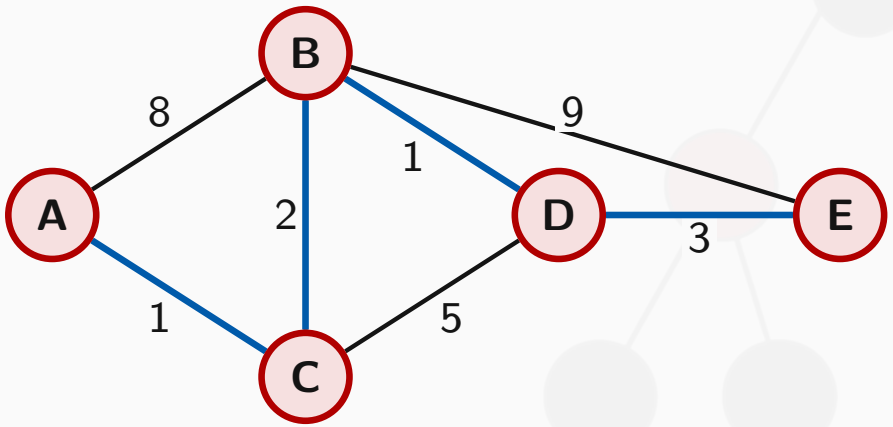



**Pop** $(7, \mathrm{E})$
No improvement. E is settled.

**Heap after this step:**

$[(8, \mathrm{B}), (12, \mathrm{E})]$

# Live 1: Step-by-Step Walkthrough

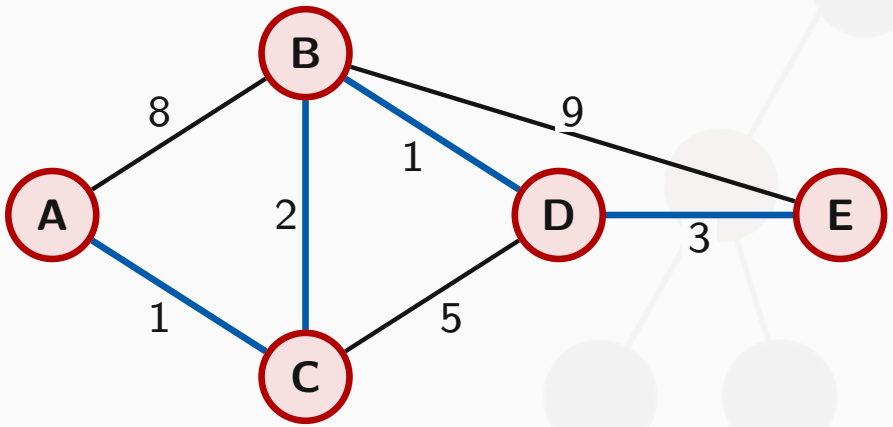



**Pop** $(8, \mathrm{B})$
Stale: $8 \neq 3$. Skip.

**Heap after this step:**

$[(12, \mathrm{E})]$

# Live 1: Step-by-Step Walkthrough

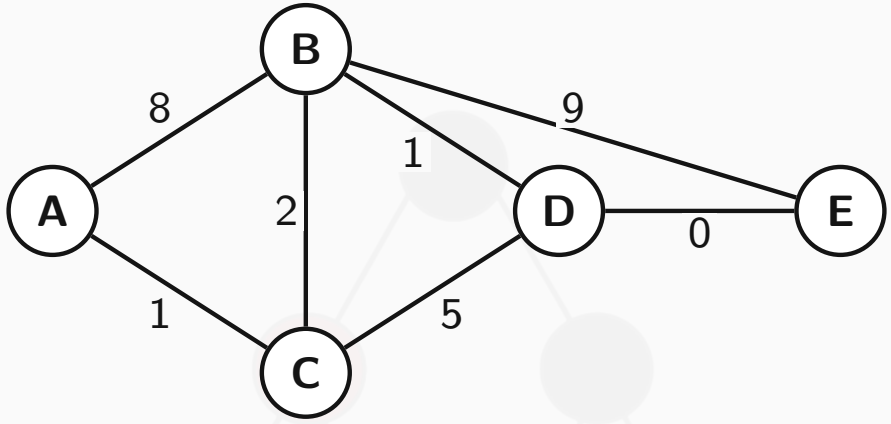



**Pop** $(12, \mathrm{E})$
Stale. The heap is empty.

**Heap after this step:**

[]

# Try It Yourself: A Free Final Edge




The same graph, but **D–E now costs 0**.

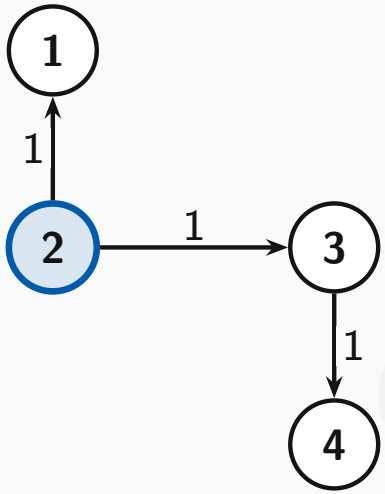
1. Find the new final distances from A and a shortest route to E.
2. When is E first discovered, and when is it safe to stop?
3. List the valid pops, breaking equal-distance ties alphabetically.

# Try It Yourself: A Free Final Edge (Answer)

- → Final distances: $[0, 3, 1, 4, 4]$. The route $A \to C \to B \to D \to E$ costs $1 + 2 + 1 + 0 = 4$.
- → E is first discovered from B with cost $3 + 9 = 12$. This is not final.
- → Valid pops: $(0, \mathrm{A}), (1, \mathrm{C}), (3, \mathrm{B}), (4, \mathrm{D}), (4, \mathrm{E})$. Equal keys use the alphabetical tie-break.
- → Stop safely at $(4, \mathrm{E})$. The remaining snapshots $(6, \mathrm{D}), (8, \mathrm{B}), (12, \mathrm{E})$ are stale.

# Live 2 / Problem 2: Network Delay Time

**LC743:** a signal starts at vertex $k$. When has *every* vertex received it? Edges are **directed** travel times.




$n = 4,\ k = 2.$
Input: $(2, 1, 1)$, $(2, 3, 1)$, $(3, 4, 1)$.
Distances in order 1–4:
$[1, 0, 1, 2]$.
Return $\max_v \mathrm{dist}[v] = \boxed{2}$.

If any vertex is unreachable, return $-1$. With a size $n + 1$ array, ignore unused index 0 in the answer.



# 0–1 BFS

# Binary Costs Do Not Need a Heap

Suppose every edge costs **0 or 1**.

- → From a route of cost $d$, the next cost is only $d$ or $d + 1$.
- → A **deque** lets us insert at either end and pop from the front.
- → A cost-0 improvement goes to the **front**: same cost layer.
- → A cost-1 improvement goes to the **back**: next cost layer.

This is **0–1 BFS**: Dijkstra's ordering, without a heap. Ordinary FIFO BFS would count edges, not total cost.

# The Deque Invariant

**Store snapshots** $(d, u)$, just as in the heap.

$$\underbrace{d,\ d,\ d}_{\text{front level}} \quad \underbrace{d+1,\ d+1}_{\text{next level}}$$

- → Stored keys are nondecreasing, with at most **two adjacent levels**.
- → On a current pop of cost $d$, front-inserting $d$ or back-inserting $d+1$ preserves this order.
- → Skip a pop when $d \neq \mathrm{dist}[u]$, exactly as with Dijkstra.

Do not confuse *stored keys* with the changing `dist` labels.

# 0–1 BFS in Code (C++ and Python)

## C++ (deque)

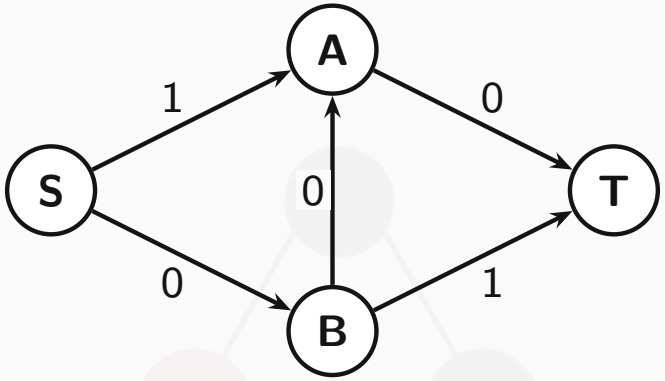
```cpp
vector<ll> dist(n, INF);
dist[src] = 0;
deque<PII> dq{{0, src}};
while (!dq.empty()) {
  auto [d, u] = dq.front();
  dq.pop_front();
  if (d != dist[u]) continue;
  for (auto [v, w] : adj[u]) {
    if (d + w < dist[v]) {
      dist[v] = d + w;
      if (w == 0)
        dq.push_front({dist[v], v});
      else
        dq.push_back({dist[v], v});
    }
  }
}
```

## Python (collections.deque)

```python
def zero_one_bfs(adj, src):
  dist = [float("inf")] * len(adj)
  dist[src] = 0
  dq = deque([(0, src)])
  while dq:
    d, u = dq.popleft()
    if d != dist[u]:
      continue
    for v, w in adj[u]:
      nd = d + w
      if nd < dist[v]:
        dist[v] = nd
        if w == 0:
          dq.appendleft((nd, v))
        else:
          dq.append((nd, v))
  return dist
```

Strict $<$ queues only genuine improvements. $O(V + E)$ time and space.

# Live: a Zero-Cost Detour Beats Discovery

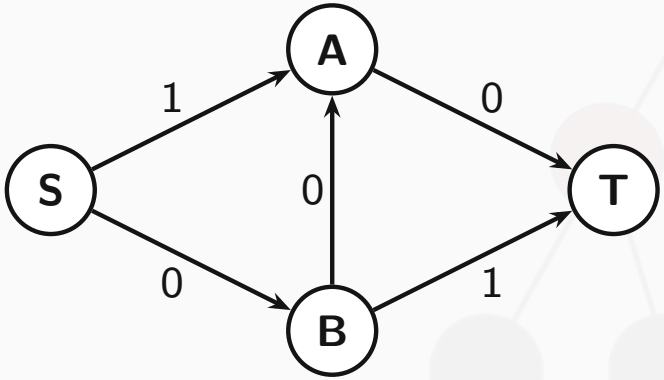



**Directed.** Scan outgoing edges in exactly this order:

$$S : [(A, 1), (B, 0)] \quad A : [(T, 0)]$$
$$B : [(A, 0), (T, 1)] \quad T : []$$

A is first discovered at 1, but $S \to B \to A$ costs 0. Predict what happens to the *old* entry for A.

# Live: Step-by-Step Deque Trace




**Initialise**

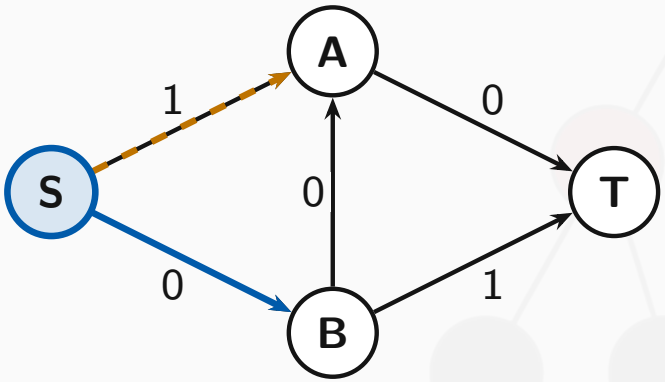
Push $(0, \mathrm{S})$.

**Deque after this step**

(front left):

$[(0, \mathrm{S})]$

# Live: Step-by-Step Deque Trace



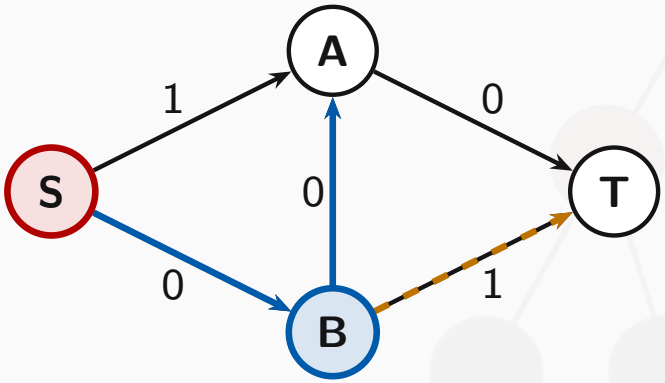

**Pop** $(0, \mathrm{S})$
B gets 0 (front). A gets 1 (back).

**Deque after this step**

(front left):

$[(0, \mathrm{B}), (1, \mathrm{A})]$

# Live: Step-by-Step Deque Trace



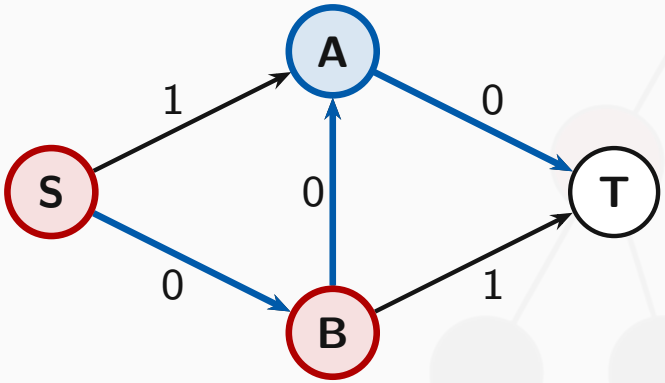

**Pop** $(0, \mathrm{B})$
A improves to 0 (front). T gets 1 (back).

**Deque after this step**

(front left):

$[(0, \mathrm{A}), (1, \mathrm{A}), (1, \mathrm{T})]$

# Live: Step-by-Step Deque Trace



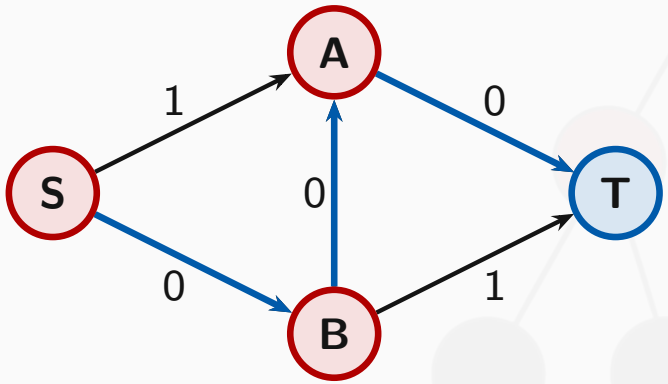

**Pop** $(0, \mathrm{A})$
T improves to 0 (front).

**Deque after this step**

(front left):

$[(0, \mathrm{T}), (1, \mathrm{A}), (1, \mathrm{T})]$

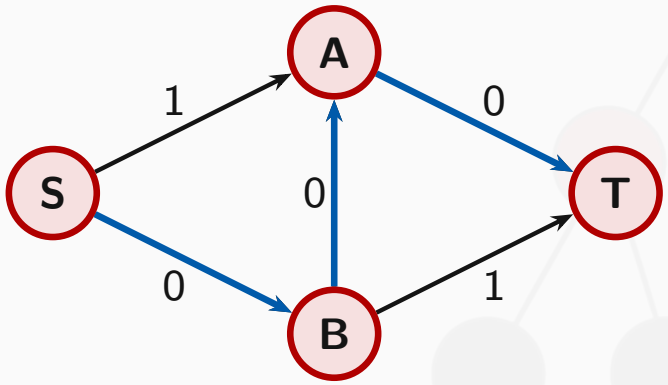
# Live: Step-by-Step Deque Trace




**Pop** $(0, \mathrm{T})$
No outgoing edges. T is settled.

**Deque after this step**
(front left):
$[(1, \mathrm{A}), (1, \mathrm{T})]$

# Live: Step-by-Step Deque Trace



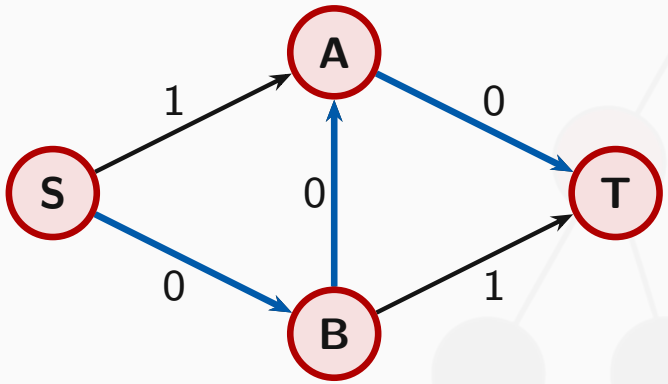

**Pop** $(1, \mathrm{A})$
Stale: skip.

**Deque after this step**

(front left):

$[(1, \mathrm{T})]$

# Live: Step-by-Step Deque Trace




**Pop** $(1, \mathrm{T})$
Stale: skip. All distances are 0.

**Deque after this step**
(front left):
[]

# Try It Yourself: Two Tempting Mistakes

Use the previous directed graph and the same edge-scan order.

1. Suppose we mark a vertex visited *when enqueuing*, and never allow it to improve. What wrong distance reaches T?
2. Keep the correct distance updates, but change the stale test to always process old entries. What work is repeated?
3. Why must the relaxation test use $<$, not $\leq$, if a zero-cost cycle exists?

Explain the role of **distance improvements**, not just queue order.

# Try It Yourself: Two Tempting Mistakes (Answer)

1. With enqueue-time visited flags, S fixes A at 1 and B at 0. B cannot improve A, and discovers T at 1. A cannot improve T either. The wrong answer is **1**, instead of 0.
2. Without the stale check, old $(1, A)$ rescans A's outgoing edge, and old $(1, T)$ processes T again. Strict relaxation prevents new improvements here, but these scans are unnecessary.
3. With $\leq$, an edge on a zero-cost cycle can requeue a vertex at the *same* distance forever. Strict $<$ queues only genuine improvements.

The correct shortest route is $S \to B \to A \to T$, total cost 0.
Discovery is not settlement: never make first discovery final.

# Path Objectives and Model Choice

# A Grid Is an Implicit Graph

- → **State**: a cell $(r, c)$; **edge**: one legal move.
- → Generate neighbours when needed: usually $(r \pm 1, c)$ and $(r, c \pm 1)$.
- → Reject out-of-bounds cells and any forbidden destinations.
- → Define the **cost of the move**, then select the algorithm.

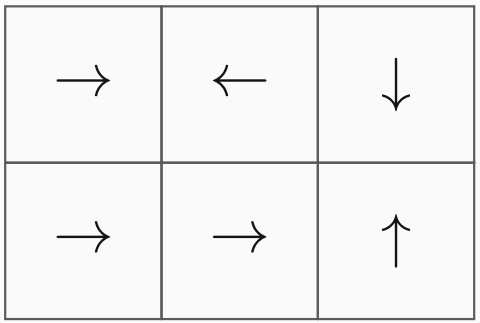
No need to build an adjacency list for every cell. An $R \times C$ four-neighbour grid has $RC$ vertices and $O(RC)$ edges.

A rectangular drawing does *not* imply unweighted BFS.

# Live 3 / Problem 3: Arrow-Grid Costs

**LC1368:** change as few cell arrows as possible to obtain a path from top left to bottom right.

start




target

Every in-bounds orthogonal move is allowed.

$$w(u,v) = \begin{cases} 0 & \text{follow the arrow at } u, \\ 1 & \text{otherwise.} \end{cases}$$

The arrow belongs to the cell we **leave**, not the one we enter.

Platform encoding: $1 = \rightarrow$, $2 = \leftarrow$, $3 = \downarrow$, $4 = \uparrow$. Use **0–1 BFS**.

# Try It Yourself: Escape the Arrow Loop

start

| | | |
|---|---|---|
| → | ← | ↓ |
| → | → | ↑ |

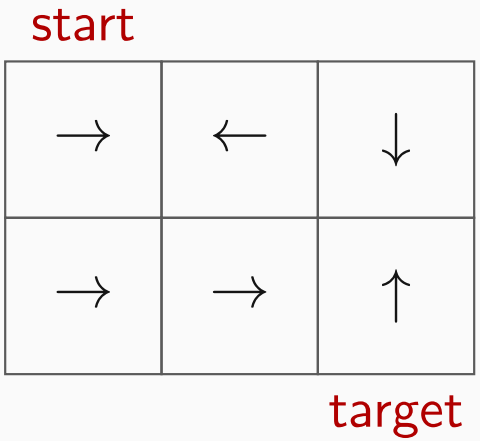
target

Input: $\begin{pmatrix} 1 & 2 & 3 \\ 1 & 1 & 4 \end{pmatrix}$.

1. What cells can we reach at cost 0 from $(0, 0)$?
2. Find a minimum-cost route to $(1, 2)$.
3. Give a lower bound proving that your route is optimal.

Following arrows forever can loop. A shortest route can be chosen without cycles, so paying for a departure corresponds to changing that cell's arrow once.

# Try It Yourself: Escape the Arrow Loop (Answer)




Zero-cost reachable cells are just $(0, 0)$ and $(0, 1)$: their arrows point to each other.
Any route to the target must leave this set, so it costs **at least 1**.

Route $(0, 0) \to (0, 1) \to (0, 2) \to (1, 2)$ has costs $0, 1, 0$. Change $(0, 1)$ from left to right. Total $\boxed{1}$ matches the lower bound.

Alternatively, change $(0, 0)$ to down, then follow the bottom row.

# Live 3: Reproducible Deque Trace

Name cells $\begin{smallmatrix} A & B & C \\ D & E & F \end{smallmatrix}$. Scan directions **right, left, down, up**; ignore out-of-bounds moves. Entries are **(stored cost, cell)**, front on the left.

| Pop | Improved labels | Deque after the pop |
|---|---|---|
| $(0, A)$ | B: 0, D: 1 | $[(0, B), (1, D)]$ |
| $(0, B)$ | C: 1, E: 1 | $[(1, D), (1, C), (1, E)]$ |
| $(1, D)$ | none | $[(1, C), (1, E)]$ |
| $(1, C)$ | F: 1 | $[(1, F), (1, E)]$ |
| $(1, F)$ | none | $[(1, E)]$ |
| $(1, E)$ | none | empty |

Complete distances:

$\begin{pmatrix} 0 & 0 & 1 \\ 1 & 1 & 1 \end{pmatrix}$. For F alone, stop at its current pop $(1, F)$. The zero cycle A–B creates no new entries because updates are strict.

# Try It Yourself: What Are We Optimising?

A route's score can be extended by a sum, a product or a maximum. Compare the candidate routes:

1. Route A has step differences $2, 9$; route B has $6, 6$. Which is better under the **sum**? Which under the **maximum**?
2. A direct route has success probability $0.75$. A two-edge route has probabilities $0.8$ and $0.9$. Which is better when we maximise the **product**?

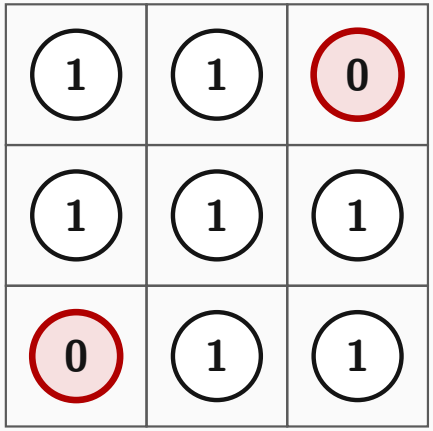
For each, state the operation used to extend a path score.

# Try It Yourself: What Are We Optimising? (Answer)

1. **Sum:** $2 + 9 = 11 < 12 = 6 + 6$, so route A wins. **Maximum:** $\max(2, 9) = 9 > 6 = \max(6, 6)$, so route B wins. Extend by *adding* for sums and by *taking a maximum* for bottlenecks.
2. **Product:** $0.8 \cdot 0.9 = 0.72 < 0.75$, so the direct route wins. Extend by *multiplying*; maximise with a max-heap.

The objective, not the graph, decides which route is best. In each case, a current best pop is final when extending a route cannot improve its score.

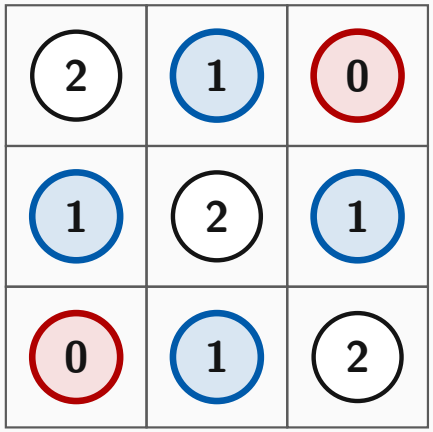
# Try It Yourself: Nearest Zero




**Multi-source BFS:** seed every zero at distance 0 and run one BFS.
Move up, down, left or right.
Coordinates are zero-based, top left is $(0, 0)$.

1. Which cells enter the queue initially, and at what distance?
2. Compute the complete distance matrix.
3. Why would a DFS's first-found route not justify these distances?

# Try It Yourself: Nearest Zero (Answer)




- → Seed $(0, 2)$ and $(2, 0)$ at distance 0 **before** expanding.
- → Layer 1: $(0, 1), (1, 0), (1, 2), (2, 1)$.
- → Layer 2: $(0, 0), (1, 1), (2, 2)$.
- → FIFO processes shorter routes first, so first discovery is minimal. DFS does not keep distance layers and may find a longer route first.
- → One BFS for all zeros: $O(RC)$ time.

# Try It Yourself: Keep the Useful Arrival

At city $u$ we can arrive in two ways:

- $\rightarrow$ cheaply but slowly: fee 3, time used 9;
- $\rightarrow$ expensively but quickly: fee 6, time used 3.

The time limit is 10. The only move from $u$ to the target takes 3 more time units, and the target's entry fee is 2. The source fee and each entered city's fee are charged.

1. Which arrival at $u$ can finish? What is its total fee?
2. Why does retaining only the cheapest fee at $u$ fail?
3. How would you label the states and reject illegal transitions?

# Try It Yourself: Keep the Useful Arrival (Answer)

| Arrival at $u$ | Time used | Finish time | Result |
|---|---|---|---|
| Cheap (fee 3) | 9 | $9+3=12$ | infeasible |
| Fast (fee 6) | 3 | $3+3=6\le 10$ | total fee $6+2=\boxed{8}$ |

→ Keeping only fee 3 at $u$ discards the arrival that can finish.

→ Let $\mathrm{dist}[u][t]$ be the minimum fee to reach $u$ at time $t$. A move taking $\tau$ tries state $(v, t+\tau)$ with fee $\mathrm{dist}[u][t]+\mathrm{fee}[v]$.

Reject $t+\tau>10$. Initially $\mathrm{dist}[s][0]=\mathrm{fee}[s]$. Minimise over feasible arrival times at the target.



# Conclusion

# Summary: Matching Models to Algorithms

| **Need / edge costs** | **First choice** | **Time** |
|---|---|---|
| Reachability only | DFS or BFS | $O(V + E)$ |
| Unit / equal positive cost | BFS | $O(V + E)$ |
| Costs 0 or 1 | 0–1 BFS | $O(V + E)$ |
| General nonnegative costs | Dijkstra | heap logarithm |
| General negative costs | e.g. Bellman–Ford | outside today's scope |

Multiple starts? Seed **all sources** at their correct initial scores.

Time, fuel, keys or remaining moves affect future choices? They may belong in the **state**, not just in an edge weight.